Frontend–Backend Integration Contract

Scope and source of truth

This document describes the behavior implemented by the FastAPI application and exercised by its tests. Executable routers, Pydantic schemas, services, repositories, models, and tests are authoritative; architecture prose and route summaries are not treated as contracts where they disagree with code.

Verified surface: **152 HTTP operations and 1 WebSocket operation**.

Group	HTTP count
Health	3
Admin and identity	20
Auth	12
Brand	5
Branch	6
Cart	7
Checkout	2
Coupon	9
Customer	10
Inventory	6
Kitchen	5
Menu	16
Merchant and staff	11
Notification	6
Order	5
Payment	9
Reports	5
Review	8
Search	2

Group	HTTP count
Subscription	5
Total	**152**

The count is generated from the FastAPI OpenAPI schema, including three root health operations and all included routers. The WebSocket in `app/order/router.py` is counted separately.

Connection, URLs, and serialization

- The backend does not configure a hostname. Frontends must receive it from deployment configuration, for example `http://localhost:8000`.
- API base path: `{BACKEND_ORIGIN}/api/v1`.
- Health paths are root-relative, not under `/api/v1`.
- OpenAPI: `/api/v1/openapi.json`; Swagger UI: `/api/v1/docs`; ReDoc: `/api/v1/redoc`.
- WebSocket base uses the matching transport and host: `ws://host/api/v1` or `wss://host/api/v1`.
- JSON is used for request and response bodies except the payment webhook, whose raw body is signature-verified before JSON parsing.
- UUID fields are JSON strings. Datetimes are ISO-8601 strings. Times are normalized to `HH:MM:SS`.
- Monetary schema fields are JSON numbers (`float`), although persistence commonly uses `NUMERIC(10,2)`.
- `204 No Content` operations have no response body.

CORS

`app/main.py` installs `CORSMiddleware` with explicit origins from `CORS_ORIGINS`, credentials enabled, and all methods and headers allowed. Development defaults permit the two Vite localhost origins. Production startup rejects an empty list or wildcard origin.

Production also rejects the committed development signing/cleanup keys, short secrets, and the mock payment provider. Configure `JWT_SECRET_KEY`, `SECRET_KEY`, `INTERNAL_CLEANUP_KEY`, JSON-array `CORS_ORIGINS`, `PAYMENT_PROVIDER=razorpay`, `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, and `RAZORPAY_WEBHOOK_SECRET` through the deployment secret/configuration system. Never expose server keys as `VITE_*` values.

Authentication and authorization

Send access tokens as:

```
Authorization: Bearer <access_token>
```

Registration and login return HS256 JWT access and refresh tokens. Access tokens last 15 minutes; refresh tokens last 7 days. Refresh tokens are bound to persisted sessions, rotated on refresh, and revoked on logout, logout-all, password reset, deactivation, or administrator action.

The canonical role vocabulary is `Customer`, `KitchenStaff`, `BranchManager`, `MerchantOwner`, and `PlatformAdmin`. Legacy `Admin` claims normalize to `PlatformAdmin`; new persistence and responses use canonical values.

Auth labels used below:

- **Public**: no bearer dependency.
- **Bearer**: any structurally valid access token; no role restriction.
- A role list is an exact, case-sensitive `require_role` check.

- Service ownership checks, where implemented, are stated separately.

Public registration always creates `Customer`; any supplied privileged role is rejected. Platform administrators can provision users and issue invitations. Merchant owners can invite `KitchenStaff` or `BranchManager` users only to branches belonging to their merchant.

Product frontend implementation requirements

The repository includes a React/Vite product application in `frontend/`. Its default experience is role-aware and API-backed; the generic OpenAPI workbench remains available to Platform Admin users at `/dev/api`.

Runtime stack and configuration

Concern	Frontend requirement
Runtime	React 19 with TypeScript and Vite
Routing	React Router with public, authentication, Customer, Merchant, Kitchen, and Platform Admin layouts
Server state	TanStack Query for loading, caching, mutation invalidation, polling, and retry
API origin	`VITE_API_BASE_URL`; leave empty during local development to use Vite's `/api` and `/health` proxy
Authentication	Access token held in memory; refresh token in tab-scoped `sessionStorage`; one refresh-and-retry after HTTP 401
Serialization	JSON request bodies, snake_case fields, UUID strings, ISO-8601 datetimes, `HH:MM:SS` branch times
Errors	Normalize both the application envelope and FastAPI's `detail` response; retain correlation ID where supplied
Real time	Branch-order WebSocket plus 10-second polling fallback and query invalidation
Responsive UX	Public top navigation, desktop workspace sidebar, compact mobile workspace navigation, accessible form labels and status states
Developer tools	Existing OpenAPI request workbench at `/dev/api`, guarded to `PlatformAdmin` in the frontend

Local development requires both processes:

```
alembic upgrade head
python scripts/create_platform_admin.py admin@example.com
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
cd frontend
npm install
npm run dev
```

The frontend then runs at `http://127.0.0.1:5173`. The API defaults to `http://127.0.0.1:8000` through the Vite proxy.

Authentication and role routing

Public registration must submit `role: "Customer"` regardless of client input. Privileged roles are created through Platform Admin provisioning/invitations or Merchant Owner staff invitations. Invitation acceptance sets the initial password, creates or activates the user, and establishes the scoped staff membership.

Backend role	Landing route	Product workspace
`Customer`	`/customer`	Profile, delivery addresses, favorites, cart, checkout, orders, payment status, notifications, reviews
`MerchantOwner`	`/merchant`	Onboarding, brands, branches, kitchens, menu, inventory, orders, reviews, subscriptions, commissions, invoices, payment lookup/refund
`KitchenStaff`	`/kitchen`	Remembered branch selection, live order board, order lookup
`BranchManager`	`/branch-manager`	Assigned-branch settings, kitchens, menu, inventory, orders, payments, and reports
`PlatformAdmin`	`/admin`	Analytics, merchant and user provisioning, review moderation, banners, coupons, feature flags, payment/refund operations, audit logs, reports, health, developer API

Frontend route guards are navigation controls only. Every protected backend operation must continue to enforce authorization server-side.

Public and Customer pages

Frontend route	Required backend operations
`/`	`GET /admin/banners`, `GET /branches`
`/discover`	`GET /brands`, `GET /branches?brand_id=`
`/brands`	`GET /brands`
`/search`	`GET /search`; suggestions can use `GET /search/suggestions`
`/coupons`	`GET /coupons`
`/branches/{branch_id}`	`GET /branches/{id}`, menu, reviews, rating summary; Customer actions use cart and favorites
`/auth`	Login or Customer-only registration
`/customer`	`GET /cart`, `GET /orders`
`/customer/profile`	`GET/PUT /customers/me`
`/customer/addresses`	Address list/create/set-primary/delete

Frontend route	Required backend operations
`/customer/favorites`	Favorites list/delete plus public branch lookup
`/customer/cart`	Cart get/update/remove/clear; server remains source of truth
`/customer/checkout`	Addresses, cart, `POST /checkout/summary`, then `POST /orders`
`/customer/orders`	Customer-scoped `GET /orders`
`/customer/orders/{order_id}`	Order detail, valid cancellation, completed-order review
`/customer/payments/new`	Payment intent and server verification using provider-returned values
`/customer/notifications`	`GET /notifications/me`

Customer UX rules:

1. Adding a menu item requires a Customer session and an existing customer profile.
2. A cart may contain items from only one branch. Display `CROSS_BRANCH_CART_CONFLICT` as a choice to clear the current cart or keep it.
3. Checkout returning HTTP 200 with `is_valid=false` is a blocked checkout, not a successful order. Render every item in `issues`.
4. Send the selected `delivery_address_id` when creating a quote. The server verifies ownership and snapshots the persisted address into the quote and order.
5. Create an order with the short-lived `quote_id`; subtotal, coupon discount, tax, delivery fee, currency, and final amount are persisted from the server quote.
6. Never calculate payment signatures or expose gateway secrets in browser code.
7. Allow cancellation only from `PLACED` or `ACCEPTED`; allow review only at `COMPLETED`.

Merchant and Kitchen pages

Frontend route	Required backend operations
`/merchant`	Merchant profile plus owned brands and branches
`/merchant/onboarding`	`GET /merchants/me`, onboard or update merchant
`/merchant/brands`	Brand list/create; update/delete when exposed by the page
`/merchant/branches`	Branch list/create/status; update/delete when exposed by the page
`/merchant/kitchens`	Branch picker and kitchen list/create
`/merchant/menu`	Public menu read and menu/category/item creation
`/merchant/inventory`	Branch inventory list/create and stock updates
`/merchant/orders`	Branch order list/filter and legal status transitions
`/merchant/reviews`	Branch reviews and merchant replies
`/merchant/coupons`	Assigned-branch coupon lifecycle and redemption history

Frontend route	Required backend operations
`/merchant/subscription`	Subscription get/create/renew, invoices, commissions
`/merchant/payments`	Tenant-scoped paginated payment history and idempotent gateway-backed refunds
`/kitchen`	Branch picker, branch orders, WebSocket invalidation, polling fallback
`/kitchen/orders/{order_id}`	Authorized assigned-branch order lookup

The legal order transition buttons are:

```
PLACED -> ACCEPTED or CANCELLED
ACCEPTED -> PREPARING or CANCELLED
PREPARING -> READY
READY -> COMPLETED
```

`GET /merchants/me/branches` returns only owned or assigned branches. Merchant staff management creates, updates, and removes persisted branch assignments. A WebSocket message is a refresh signal; refetch orders after every event and after reconnecting.

Platform Admin pages

Frontend route	Required backend operations
`/admin`	`GET /admin/analytics/overview`
`/admin/approvals`	Filterable merchant queue, detail, approval/rejection, and valid status transitions
`/admin/users`	User provisioning, role/status updates, invitations, and session revocation
`/admin/moderation`	Hidden-review queue and audited moderation
`/admin/reports`	Platform operational, payment/refund, and commission reporting
`/admin/banners`	Banner list/create/delete
`/admin/coupons`	Platform or branch coupon creation
`/admin/flags`	Feature flag list/update
`/admin/refunds`	Synthetic refund override record
`/admin/audit`	Audit log list
`/admin/status`	Public health/live/readiness probes
`/dev/api`	Live OpenAPI catalog and request workbench

Admin UX must identify remaining prototype behavior accurately:

- Refund override produces an audit record and synthetic identifier; it does not call Razorpay.

- Feature flags are persisted, but they are not yet a distributed dynamic-config system and not every flag is enforced in request paths.

Shared frontend data and interaction requirements

- Keep domain DTOs typed in `frontend/src/api/types.ts`; isolate transport calls in `frontend/src/api/modules.ts` and normalize failures in `frontend/src/api/client.ts`.
- Render explicit loading, empty, success, validation, disconnected, and error states.
- Do not use hidden navigation as a security boundary.
- Invalidate related queries after every mutation: cart, addresses, favorites, merchant entities, menus, inventory, orders, reviews, subscriptions, banners, flags, and merchant status.
- Mask bank account values in presentation and never place access tokens, refresh tokens, passwords, payment signatures, or full bank details in logs/history.
- Treat monetary values returned as JSON numbers as display values; format as INR but avoid recomputing authoritative totals in the browser.
- Keep destructive operations visually distinct and require confirmation before production use.
- Preserve correlation IDs in user-visible error details for support diagnostics.
- Test role redirects, token refresh, API error normalization, cart constraints, checkout blocking, legal status transitions, and branch selection persistence.

HTTP endpoint catalog

All paths in this section are exact.

Health (3)

Method and path	Auth	Request	Success response
`GET /health`	Public	None	`200 {"status":"ok","version":string}`
`GET /health/live`	Public	None	`200 {"status":"alive"}`
`GET /health/ready`	Public	None	`200 {"status":"ready","database":"connected"}` or `503` when disconnected

Authentication (12)

Method and path	Auth	Request	Success response
`POST /api/v1/auth/register`	Public	`UserRegisterRequest`	`201 TokenResponse`
`POST /api/v1/auth/login`	Public	`UserLoginRequest`	`200 TokenResponse`
`POST /api/v1/auth/refresh`	Public	`TokenRefreshRequest`	`200 TokenResponse`
`GET /api/v1/auth/me`	Bearer	None	`200 UserResponse`
`POST /api/v1/auth/verify-email`	Public	token	`200 MessageResponse`

Method and path	Auth	Request	Success response
`POST /api/v1/auth/verification/resend`	Public	email	`200 MessageResponse`
`POST /api/v1/auth/password/forgot`	Public	email	`200 MessageResponse`
`POST /api/v1/auth/password/reset`	Public	token and new password	`200 MessageResponse`
`POST /api/v1/auth/password/change`	Bearer	current and new password	`200 MessageResponse`
`POST /api/v1/auth/logout`	Public	refresh token	`200 MessageResponse`
`POST /api/v1/auth/logout-all`	Bearer	None	`200 MessageResponse`
`POST /api/v1/auth/invitations/accept`	Public	token and password	`201 TokenResponse`

Registration lowercases and trims email, hashes the password with Argon2id, creates an active but unverified user, and issues tokens. Login rejects bad credentials and inactive users. Refresh requires a refresh-token `type`, an existing active user, and issues a new pair.

Admin and identity (20)

Method and path	Auth	Request/query	Success response
`POST /api/v1/admin/merchants/{merchant_id}/approval`	`PlatformAdmin`	UUID path; `MerchantApprovalRequest`	`200 {"merchant_id":string,"status":"APPROVE`
`POST /api/v1/admin/coupons`	`PlatformAdmin`	`CouponCreate`	`201 CouponResponse`
`POST /api/v1/admin/banners`	`PlatformAdmin`	`BannerCreate`	`201 BannerResponse`
`GET /api/v1/admin/banners`	Public	`active_only:boolean=true`	`200 BannerResponse[]`
`DELETE /api/v1/admin/banners/{banner_id}`	`PlatformAdmin`	UUID path	`204`
`GET /api/v1/admin/feature-flags`	`PlatformAdmin`	None	`200 FeatureFlagResponse[]`
`PUT /api/v1/admin/feature-flags/{flag_key}`	`PlatformAdmin`	string path; `FeatureFlagUpdate`	`200 FeatureFlagResponse`
`POST /api/v1/admin/refunds/override`	`PlatformAdmin`	`RefundOverrideRequest`	`200 RefundOverrideResponse`
`GET /api/v1/admin/audit-logs`	`PlatformAdmin`	`limit:int=50`, 1–500	`200 AdminAuditLogResponse[]`
`GET /api/v1/admin/analytics/overview`	`PlatformAdmin`	None	`200 PlatformAnalyticsOverview`

Method and path	Auth	Request/query	Success response
`GET /api/v1/admin/merchants`	`PlatformAdmin`	status/search/pagination filters	`200 MerchantAdminPage`
`GET /api/v1/admin/merchants/{merchant_id}`	`PlatformAdmin`	UUID path	`200 MerchantAdminSummary`
`PATCH /api/v1/admin/banners/{banner_id}`	`PlatformAdmin`	`BannerUpdate`	`200 BannerResponse`
`GET /api/v1/admin/users`	`PlatformAdmin`	skip/limit	`200 UserResponse[]`
`POST /api/v1/admin/users`	`PlatformAdmin`	user create payload	`201 UserResponse`
`PATCH /api/v1/admin/users/{user_id}`	`PlatformAdmin`	role/status update	`200 UserResponse`
`POST /api/v1/admin/users/{user_id}/deactivate`	`PlatformAdmin`	None	`200 UserResponse`
`GET /api/v1/admin/users/{user_id}/sessions`	`PlatformAdmin`	UUID path	`200 SessionResponse[]`
`DELETE /api/v1/admin/users/{user_id}/sessions`	`PlatformAdmin`	UUID path	`200 MessageResponse`
`POST /api/v1/admin/invitations`	`PlatformAdmin`	invitation payload	`201 InvitationResponse`
`GET /api/v1/admin/reviews/hidden`	`PlatformAdmin`	None	`200 ReviewResponse[]`

Merchant queue/detail, approval, user lifecycle, invitations, session revocation, hidden-review moderation support, banners, durable feature flags, reports, audit logs, and analytics are Platform Admin operations. Admin refund override creates an audit record and synthetic identifier only; it does not call the payment gateway or append a payment transaction.

Brands (5)

Method and path	Auth	Request/query	Success response
`GET /api/v1/brands`	Public	`merchant_id?:string`; `skip:int=0` ≥0; `limit:int=100` 1–100	`200 BrandResponse[]`
`POST /api/v1/brands`	`MerchantOwner`, `PlatformAdmin`	`BrandCreateRequest`	`201 BrandResponse`
`GET /api/v1/brands/{brand_id}`	Public	UUID-formatted string path	`200 BrandResponse`
`PATCH /api/v1/brands/{brand_id}`	`MerchantOwner`, `PlatformAdmin`	UUID-formatted string; `BrandUpdateRequest`	`200 BrandResponse`
`DELETE /api/v1/brands/{brand_id}`	`MerchantOwner`, `PlatformAdmin`	UUID-formatted string	`204`

Non-admin create/update/delete checks ownership through the parent merchant. Reads return only repository-visible, non-deleted records.

Branches (6)

Method and path	Auth	Request/query	Success response
`GET /api/v1/branches`	Public	`brand_id?:string`; `skip:int=0` ≥0; `limit:int=100` 1–100	`200 BranchResponse[]`
`POST /api/v1/branches`	`MerchantOwner`, `PlatformAdmin`	`BranchCreateRequest`	`201 BranchResponse`
`GET /api/v1/branches/{branch_id}`	Public	UUID-formatted string	`200 BranchResponse`
`PATCH /api/v1/branches/{branch_id}`	`BranchManager`, `MerchantOwner`, `PlatformAdmin`	UUID-formatted string; `BranchUpdateRequest`	`200 BranchResponse`
`PATCH /api/v1/branches/{branch_id}/status`	Same	UUID-formatted string; `BranchStatusUpdateRequest`	`200 BranchResponse`
`DELETE /api/v1/branches/{branch_id}`	`MerchantOwner`, `PlatformAdmin`	UUID-formatted string	`204`

Creation is limited to Merchant Owner or Platform Admin. Update/status authorize the owning merchant or an active branch assignment; deletion remains owner/admin only.

Kitchens (5)

Method and path	Auth	Request/query	Success response
`GET /api/v1/branches/{branch_id}/kitchens`	`KitchenStaff`, `BranchManager`, `MerchantOwner`, `PlatformAdmin`	UUID-formatted string; `skip:int=0` ≥0; `limit:int=100` 1–100	`200 KitchenResponse[]`
`POST /api/v1/kitchens`	`BranchManager`, `MerchantOwner`, `PlatformAdmin`	`KitchenCreateRequest`	`201 KitchenResponse`
`GET /api/v1/kitchens/{kitchen_id}`	`KitchenStaff`, `BranchManager`, `MerchantOwner`, `PlatformAdmin`	UUID-formatted string	`200 KitchenResponse`
`PATCH /api/v1/kitchens/{kitchen_id}`	`BranchManager`, `MerchantOwner`, `PlatformAdmin`	UUID-formatted string; `KitchenUpdateRequest`	`200 KitchenResponse`
`DELETE /api/v1/kitchens/{kitchen_id}`	`BranchManager`, `MerchantOwner`, `PlatformAdmin`	UUID-formatted string	`204`

All kitchen reads and mutations resolve the kitchen branch and enforce merchant ownership or an active staff branch assignment. Platform Admin bypass is explicit.

Menus (16)

Method and path	Auth	Request	Success response
`GET /api/v1/branches/{branch_id}/menu`	Public	UUID path	`200 MenuResponse`
`POST /api/v1/branches/{branch_id}/menus`	`BranchManager`, `MerchantOwner`, `PlatformAdmin`	UUID path; `MenuCreate`	`201 MenuResponse`
`POST /api/v1/branches/{branch_id}/menu/categories`	Same	UUID path; `MenuCategoryCreate`	`201 MenuCategoryResponse`
`POST /api/v1/branches/{branch_id}/menu/items`	Same	UUID path; `MenuItemCreate`	`201 MenuItemResponse`
`GET /api/v1/menu/items/{item_id}`	Public	UUID path	`200 MenuItemResponse`
`PATCH /api/v1/menu/items/{item_id}`	`BranchManager`, `MerchantOwner`, `PlatformAdmin`	UUID path; `MenuItemUpdate`	`200 MenuItemResponse`

The remaining operations are menu update; category get/update/delete; item availability/delete; and modifier create/get/update/delete. Every mutation resolves the parent branch and applies branch authorization. Public reads expose catalog details; mutation roles are Branch Manager, Merchant Owner, and Platform Admin.

Inventory (6)

Method and path	Auth	Request	Success response
`GET /api/v1/branches/{branch_id}/inventory`	`KitchenStaff`, `BranchManager`, `MerchantOwner`, `PlatformAdmin`	UUID path	`200 InventoryItemResponse[]`
`POST /api/v1/branches/{branch_id}/inventory`	Same	UUID path; `InventoryItemCreateRequest`	`201 InventoryItemResponse`
`PATCH /api/v1/inventory/{id}/stock`	Same	UUID path; `StockUpdateRequest`	`200 InventoryItemResponse`
`GET /api/v1/branches/{branch_id}/inventory/details`	Same	UUID path	`200 InventoryItemDetailResponse[]`
`GET /api/v1/inventory/{id}`	Same	UUID path	`200 InventoryItemDetailResponse`
`GET /api/v1/inventory/{id}/logs`	Same	UUID path; page/page_size	`200 InventoryLogPage`

Every inventory operation enforces ownership or an active staff assignment for the resource branch. Stock writes reject negative outcomes and append immutable logs.

Customers (9)

Method and path	Auth	Request	Success response
`GET /api/v1/customers/me`	Bearer	None	`200 CustomerProfileResponse`

Method and path	Auth	Request	Success response
`PUT /api/v1/customers/me`	Bearer	`CustomerProfileUpdate`	`200 CustomerProfileResponse`
`GET /api/v1/customers/me/addresses`	Bearer	None	`200 CustomerAddressResponse[]`
`POST /api/v1/customers/me/addresses`	Bearer	`CustomerAddressCreate`	`201 CustomerAddressResponse`
`PUT /api/v1/customers/me/addresses/{address_id}/primary`	Bearer	UUID path	`200 CustomerAddressResponse`
`DELETE /api/v1/customers/me/addresses/{address_id}`	Bearer	UUID path	`204`
`GET /api/v1/customers/me/favorites`	Bearer	None	`200 CustomerFavoriteResponse[]`
`POST /api/v1/customers/me/favorites`	Bearer	`CustomerFavoriteCreate`	`201 CustomerFavoriteResponse`
`DELETE /api/v1/customers/me/favorites/{branch_id}`	Bearer	UUID path	`204`

Customer routes require the Customer role and scope addresses/favorites to the authenticated profile. Address update is the tenth operation.

Cart (7)

Method and path	Auth	Request/query	Success response
`GET /api/v1/cart`	Bearer	None	`200 CartResponse`
`POST /api/v1/cart/items`	Bearer	`CartItemAddRequest`	`200 CartResponse`
`PUT /api/v1/cart/items/{item_id}`	Bearer	UUID path; `CartItemUpdateRequest`	`200 CartResponse`
`DELETE /api/v1/cart/items/{item_id}`	Bearer	UUID path	`200 CartResponse`
`DELETE /api/v1/cart`	Bearer	None	`204`
`POST /api/v1/cart/validate`	Bearer	None	`200 CartValidationResponse`
`DELETE /api/v1/cart/purge-expired`	Internal cleanup key	`max_age_days:int=7`, 1–30	`200 {"purged_count":int,"max_age_days":int}`

Cart use requires a Customer profile. A cart may contain items from one branch only. The cleanup route is server-only and requires the configured internal cleanup key; it intentionally has no product GUI.

Checkout (2)

Method and path	Auth	Request	Success response
`POST /api/v1/checkout/validate`	`Customer`	`CheckoutValidateRequest`	`200 CheckoutValidationResponse`
`POST /api/v1/checkout/summary`	`Customer`	Same	`200 CheckoutValidationResponse`

Both routes issue the same short-lived persisted quote. Validation checks customer and cart ownership, address ownership, branch/kitchen availability, menu hierarchy, inventory, and coupon validity. It snapshots address/items and computes the trusted subtotal, discount, tax, delivery fee, currency, and final amount. Business issues return `200` with `is_valid=false`; valid responses include `quote_id` and expiry.

Coupons (9)

Method and path	Auth	Request/query	Success response
`POST /api/v1/coupons/validate`	Bearer	`CouponValidateRequest`	`200 CouponValidateResponse`
`POST /api/v1/coupons`	Merchant/branch manager or `PlatformAdmin`	`CouponCreate`	`201 CouponResponse`
`GET /api/v1/coupons`	Public	`branch_id?:UUID`	`200 CouponResponse[]`
`GET /api/v1/coupons/{coupon_id}`	Public	UUID path	`200 CouponResponse`
`PATCH /api/v1/coupons/{coupon_id}`	Tenant manager/admin	`CouponUpdate`	`200 CouponResponse`
`POST /api/v1/coupons/{coupon_id}/activate`	Tenant manager/admin	UUID path	`200 CouponResponse`
`POST /api/v1/coupons/{coupon_id}/deactivate`	Tenant manager/admin	UUID path	`200 CouponResponse`
`DELETE /api/v1/coupons/{coupon_id}`	Tenant manager/admin	UUID path	`204`
`GET /api/v1/coupons/{coupon_id}/redemptions`	Tenant manager/admin	UUID path	`200 CouponRedemptionResponse[]`

Global coupons are Platform Admin-only; merchant coupons require an authorized branch. Date order, discount bounds, branch scope, and per-customer redemption limits are validated. Redemption is atomically recorded when a quoted order is created.

Orders (5 HTTP)

Method and path	Auth	Request/query	Success response
`POST /api/v1/orders`	Bearer	`OrderCreateRequest`	`201 OrderResponse`
`GET /api/v1/orders`	Bearer	`branch_id?:UUID`; query key `status?:string`	`200 OrderResponse[]`

Method and path	Auth	Request/query	Success response
`GET /api/v1/orders/{id}`	Bearer	UUID path	`200 OrderResponse`
`PATCH /api/v1/orders/{id}/status`	Bearer	UUID path; `OrderStatusUpdateRequest`	`200 OrderResponse`
`POST /api/v1/orders/{id}/cancel`	Bearer	UUID path; no body	`200 OrderResponse`

Customer order creation consumes an owned, unexpired, single-use checkout quote under a row lock. In one transaction it persists financial/address/item snapshots, reserves stock, records coupon redemption, consumes the quote, and clears cart items. Duplicate or foreign quote use is rejected. Cancellation restores active reservations once.

State transitions are exact and case-sensitive: `PLACED -> ACCEPTED|CANCELLED`, `ACCEPTED -> PREPARING|CANCELLED`, `PREPARING -> READY`, `READY -> COMPLETED`. Customer reads/cancellation are order-owner scoped; operational reads and transitions require branch ownership/assignment; Platform Admin has explicit access.

Payments (9)

Method and path	Auth	Request/header	Success response
`POST /api/v1/payments/intent`	Bearer	`PaymentIntentCreateRequest`	`201 PaymentIntentResponse`
`POST /api/v1/payments/verify`	Bearer	`PaymentVerifyRequest`	`200 PaymentVerifyResponse`
`POST /api/v1/payments/webhook`	Public gateway callback	Raw JSON body; required `X-Razorpay-Signature` header	`200 {"status":"processed","event":string|nu` or ignored result
`POST /api/v1/payments/{id}/refund`	`BranchManager`, `MerchantOwner`, `PlatformAdmin`	payment UUID; `RefundCreateRequest`	`201 PaymentTransactionResponse`
`GET /api/v1/payments/{id}`	Bearer	payment UUID	`200 PaymentResponse`
`POST /api/v1/payments/mock/complete`	Owning Customer; mock provider only	order UUID	`200 PaymentVerifyResponse`
`GET /api/v1/payments`	Customer or tenant-scoped operator	optional branch	`200 PaymentResponse[]`
`GET /api/v1/payments/history`	Customer or tenant-scoped operator	branch/page/page_size	`200 PaymentPage`
`GET /api/v1/payments/order/{order_id}`	Order owner or tenant-scoped operator	UUID path	`200 PaymentResponse`

Detailed payment flow is below.

Reviews (8)

Method and path	Auth	Request/query	Success response
`POST /api/v1/reviews`	Bearer	`ReviewCreate`	`201 ReviewResponse`

Method and path	Auth	Request/query	Success response
`GET /api/v1/branches/{branch_id}/reviews`	Public	UUID path; `include_hidden:boolean=false`	`200 ReviewResponse[]`
`GET /api/v1/branches/{branch_id}/rating-summary`	Public	UUID path	`200 RestaurantRatingSummary`
`GET /api/v1/reviews/{review_id}`	Public	UUID path	`200 ReviewResponse`
`POST /api/v1/reviews/{review_id}/reply`	Authorized merchant/branch/admin	UUID path; `ReviewReply`	`200 ReviewResponse`
`PUT /api/v1/reviews/{review_id}/moderate`	`PlatformAdmin`	UUID path; `ReviewModerateRequest`	`200 ReviewResponse`
`DELETE /api/v1/reviews/{review_id}`	Review owner or `PlatformAdmin`	UUID path	`204`
`GET /api/v1/admin/reviews/hidden`	`PlatformAdmin`	None	`200 ReviewResponse[]`

Creation requires an owned completed order and one review per order. Public lists omit hidden reviews. Detail, replies, moderation, and deletion enforce customer ownership, tenant branch access, or Platform Admin authority as appropriate.

Search (2)

Method and path	Auth	Request/query	Success response
`GET /api/v1/search`	Public	See query contract below	`200 SearchResponse`
`GET /api/v1/search/suggestions`	Public	`q:string` required, min length 1; `limit:int=10`, 1–20	`200 SearchSuggestionsResponse`

Search query fields: `q?:string`; `type:string="all"` (documented values `all|restaurant|dish|category`, not schema-enumerated); `is_veg?:boolean`; `cuisine?:string`; `min_price?:float` ≥0; `max_price?:float` ≥0; `is_active_only:boolean=true`; `latitude?:float` -90..90; `longitude?:float` -180..180; `radius_km?:float=10.0` 0.1..100; `sort_by:string="relevance"` (documented `relevance|name|price_asc|price_desc|distance`, not schema-enumerated); `skip:int=0` ≥0; `limit:int=20` 1–100. There is no validation that min ≤ max or that latitude and longitude are supplied together.

Notifications (6)

Method and path	Auth	Request	Success response
`GET /api/v1/notifications/me`	Bearer	None	`200 NotificationResponse[]`
`POST /api/v1/notifications/send`	`PlatformAdmin`	`NotificationSendRequest`	`202 NotificationResponse`
`PATCH /api/v1/notifications/{notification_id}/read`	Owner	UUID path	`200 NotificationResponse`

Method and path	Auth	Request	Success response
`PATCH /api/v1/notifications/me/read-all`	Bearer	None	`200 {"updated":int}`
`GET /api/v1/notifications/me/preferences`	Bearer	None	`200 NotificationPreferences`
`PUT /api/v1/notifications/me/preferences`	Bearer	preferences	`200 NotificationPreferences`

The send route ignores `channel`, always creates an `EMAIL` notification, and synchronously awaits a mock email provider despite accepting `BackgroundTasks`. It returns `SENT` or `FAILED`; recipient syntax is not validated as email.

Merchants and staff (11)

Method and path	Auth	Request	Success response
`GET /api/v1/merchants/me`	`MerchantOwner`, `PlatformAdmin`	None	`200 MerchantResponse`
`POST /api/v1/merchants/onboard`	`MerchantOwner`, `PlatformAdmin`	`MerchantOnboardRequest`	`201 MerchantResponse`
`GET /api/v1/merchants/{merchant_id}`	Bearer	UUID-formatted string	`200 MerchantResponse`
`PATCH /api/v1/merchants/{merchant_id}`	`MerchantOwner`, `PlatformAdmin`	UUID-formatted string; `MerchantUpdateRequest`	`200 MerchantResponse`
`PATCH /api/v1/merchants/{merchant_id}/status`	`PlatformAdmin`	UUID-formatted string; `MerchantStatusUpdateRequest`	`200 MerchantResponse`
`GET /api/v1/merchants/me/branches`	Merchant/staff	None	`200 BranchResponse[]`
`POST /api/v1/merchants/me/submit`	`MerchantOwner`	None	`200 MerchantResponse`
`GET /api/v1/merchants/staff`	`MerchantOwner`	None	`200 StaffResponse[]`
`POST /api/v1/merchants/staff/invitations`	`MerchantOwner`	email, staff role, branch IDs	`201 InvitationResponse`
`PATCH /api/v1/merchants/staff/{membership_id}`	`MerchantOwner`	role/status/branch IDs	`200 StaffResponse`
`DELETE /api/v1/merchants/staff/{membership_id}`	`MerchantOwner`	UUID path	`204`

Onboarding permits one merchant per owner and unique non-null GSTIN, starts at `DRAFT`, and requires explicit submit/review. Staff invitations are constrained to staff roles and branches owned by the inviter's merchant. Sensitive bank values remain server data and are masked in product presentation.

Subscriptions (5)

Method and path	Auth	Request	Success response
`GET /api/v1/merchants/me/subscription`	Bearer	None	`200 MerchantSubscriptionResponse`
`POST /api/v1/merchants/me/subscription`	Bearer	`SubscriptionCreateRequest`	`201 MerchantSubscriptionResponse`
`POST /api/v1/merchants/me/subscription/renew`	Bearer	`SubscriptionRenewalRequest`	`200 MerchantSubscriptionResponse`
`GET /api/v1/merchants/me/commissions`	Bearer	None	`200 PlatformCommissionResponse[]`
`GET /api/v1/merchants/me/invoices`	Bearer	None	`200 SubscriptionInvoiceResponse[]`

All subscription operations require Merchant Owner and resolve the caller's merchant. Plans are BASIC (1 branch, 5%, 0), PROFESSIONAL (5, 3.5%, 1999), and ENTERPRISE (unlimited, 2%, 4999). Branch creation enforces the active plan limit. Invoices are currently synthesized from subscription state rather than persisted payment receipts.

Reports (5)

`GET /reports/merchant/sales`, `/reports/branches/{branch_id}/performance`, `/reports/payments-refunds`, and `/reports/commissions` require merchant/branch scope or Platform Admin as applicable. `GET /reports/platform/operations` is Platform Admin-only. All accept optional bounded date windows and return server aggregates.

Field-level JSON contracts

Notation: `required` means omission is rejected. `optional` accepts null unless stated otherwise. Unlisted extra fields follow Pydantic's default behavior and are ignored.

Auth types

- `UserRegisterRequest`: `email` required valid email; `password` required string, min 8; optional compatibility `role` may only be `Customer`.
- `UserLoginRequest`: `email` required valid email; `password` required string.
- `TokenRefreshRequest`: `refresh_token` required string.
- `UserResponse`: server fields `id:string`, `email:string`, `role:string`, `is_active:boolean`, `is_verified:boolean`, `created_at:datetime`, `updated_at:datetime`.
- `TokenResponse`: `access_token:string`, `refresh_token:string`, `token_type:string` (`bearer`), `expires_in:int` (900), `user:UserResponse`.

Admin types

- `MerchantApprovalRequest`: `action` required string, service accepts case-insensitive `APPROVE|REJECT`; `reason?:string` max 500.
- `BannerCreate`: `title` required 1–150; `image_url` required 1–255; `target_url?` max 255; `display_order:int=0` ≥0; `is_active:boolean=true`.
- `BannerResponse`: server `id:UUID`, request fields, `created_at:datetime`.
- `FeatureFlagUpdate`: `enabled` required boolean; `value?:string`.
- `FeatureFlagResponse`: `flag_key:string`, `enabled:boolean`, `value?:string`.

- `RefundOverrideRequest`: `payment_id` required UUID; `amount` required number >0; `reason` required 1–500.
- `RefundOverrideResponse`: `refund_transaction_id:string`, `payment_id:UUID`, `amount:number`, `status:string`, `reason:string`, `processed_at:datetime`.
- `AdminAuditLogResponse`: `id:UUID`, `admin_user_id:UUID`, `action:string`, `target_entity:string`, `target_entity_id?:UUID`, `changes_payload:any`, `ip_address:string`, `created_at:datetime`.
- `PlatformAnalyticsOverview`: integer `total_merchants`, `active_merchants`, `total_orders`, `total_customers`; numeric `gross_revenue`, `platform_commissions`.

Merchant, brand, branch, kitchen

- `MerchantOnboardRequest`: required `business_name` and `legal_entity_name` 2–150; required `fssai_license_number` exactly 14 digits after trim; optional `gstin` matching `^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]$` after uppercase/trim; optional `bank_account_number` max 50; optional `bank_ifsc_code` max 20.
- `MerchantUpdateRequest`: all optional; names retain 2–150 constraints; FSSAI/GSTIN use the same validators; bank fields have no max constraints here.
- `MerchantStatusUpdateRequest`: required `status`, normalized uppercase and limited to `DRAFT|PENDING_VERIFICATION|APPROVED|REJECTED|SUSPENDED`.
- `MerchantResponse`: server `id`, `owner_user_id`, names, nullable `gstin`, FSSAI, nullable bank fields, `status`, `created_at`, `updated_at`; IDs are strings.
- `BrandCreateRequest`: optional `merchant_id:string`; required `name` 2–100; optional `logo_url`/`banner_url` max 255, `description`, `cuisine_types:string[]`.
- `BrandUpdateRequest`: same mutable fields, all optional.
- `BrandResponse`: server `id:string`, `merchant_id:string`, request metadata, `created_at`, `updated_at`.
- `BranchCreateRequest`: required `brand_id:string`, `name` 2–150, `address_text` 2–255; optional `latitude` -90..90 and `longitude` -180..180; `is_active=true`; `opening_time="08:00:00"` and `closing_time="23:00:00"`, each parseable as `HH:MM` or `HH:MM:SS`.
- `BranchUpdateRequest`: all mutable fields optional with matching constraints.
- `BranchStatusUpdateRequest`: required `is_active:boolean`.
- `BranchResponse`: server IDs/names/address, nullable coordinates, active flag, normalized time strings, `created_at`, `updated_at`.
- `KitchenCreateRequest`: required `branch_id:string`; `name="Main Kitchen"` 2–150; `is_active=true`.
- `KitchenUpdateRequest`: optional `name` 2–150 and `is_active`.
- `KitchenResponse`: server `id:string`, `branch_id:string`, `name`, `is_active`, timestamps.

Menu and inventory

- `MenuCreate`: `name="Main Menu"` max 100; `is_active=true`.
- `MenuCategoryCreate`: required `name` max 100; `display_order:int=0`.
- `MenuItemCreate`: required `category_id:UUID`, `name` max 150, `base_price:number`; optional `description`, `image_url`; `is_available=true`, `is_veg=true`. `base_price` has no schema lower bound but service/database require ≥0.
- `MenuItemUpdate`: optional `name`, `description`, `base_price`, `image_url`, `is_available`, `is_veg`, `category_id`; unlike create, update name/image have no length limits; `category_id` is ignored.
- `MenuModifierResponse`: `id`, `menu_item_id`, `name` max 100, `price_delta` ≥0, timestamps. Modifier create/update/get/delete operations are implemented.
- `MenuItemResponse`: request/base fields plus `id`, `category_id`, timestamps, `modifiers:MenuModifierResponse[]`.
- `MenuCategoryResponse`: `id`, `menu_id`, `name`, `display_order`, `items:MenuItemResponse[]`, timestamps.
- `MenuResponse`: `id`, `branch_id`, `name`, `is_active`, `categories:MenuCategoryResponse[]`, timestamps.
- `InventoryItemCreateRequest`: required `menu_item_id:UUID`; `initial_quantity:int=0` ≥0.
- `StockUpdateRequest`: required `quantity:int`; optional `is_in_stock`; `reason:string="MERCHANT_RESTOCK"` max 100.
- `InventoryItemResponse`: server `id`, `branch_id`, `menu_item_id`, `quantity`, `is_in_stock`, timestamps.

Customer and cart

- `CustomerProfileUpdate`: optional `full_name` 1–100 and `phone_number` max 20.
- `CustomerProfileResponse`: `id`, `user_id`, `full_name`, nullable `phone_number`, timestamps.

- `CustomerAddressCreate`: required `address_line` 1–255 and `campus_zone` 1–100; optional `building_name` max 100; `is_primary=false`.
- `CustomerAddressResponse`: server `id`, `customer_profile_id`, create fields, timestamps.
- `CustomerFavoriteCreate`: required `branch_id:UUID`.
- `CustomerFavoriteResponse`: server `id`, `customer_profile_id`, `branch_id`, `created_at`.
- `CartItemAddRequest`: required `branch_id:UUID`, `menu_item_id:UUID`; `quantity:int=1` >0; `selected_modifiers?:object[]` with unrestricted object shape.
- `CartItemUpdateRequest`: required `quantity:int` ≥0; optional unrestricted `selected_modifiers:object[]`.
- `CartItemResponse`: server IDs, quantity/modifiers, nullable derived `menu_item_name`, `unit_price`, `subtotal`, nullable `is_available`, timestamps.
- `CartResponse`: server `id`, `customer_profile_id`, `branch_id`, `items`, derived `total_items`, `subtotal`, `is_expired`, timestamps.
- `CartValidationResponse`: `is_valid:boolean`, `issues[]` (`code`, `message`, optional `item_id`, `menu_item_id`), `subtotal`, `total_items`.

Checkout and coupon

- `CheckoutValidateRequest`: required `cart_id:string`; optional `coupon_code:string`, `delivery_address_id:string`.
- `CheckoutValidationResponse`: `is_valid`; nullable `quote_id` and `quote_expires_at`; string `customer_id`, `branch_id`, `branch_name`, `cart_id`; nullable `kitchen_id`, `coupon_code`; numeric `subtotal`, `tax_amount`, `delivery_fee`, `discount_amount`, `final_payable_amount`; `items[]`; `issues[]`.
- `CouponCreate`: required `code` 2–50, `discount_type`, `discount_value` >0, `valid_from`, `valid_until`; optional `max_discount_amount` >0 and `branch_id:UUID`; `min_order_amount=0` ≥0; `max_redemptions_per_user=1` ≥1; `is_active=true`.
- `CouponResponse`: server `id`, all normalized create fields, derived `is_platform_coupon`, timestamps.
- `CouponValidateRequest`: required `code` 2–50 and `order_amount` >0; optional `branch_id:UUID`.
- `CouponValidateResponse`: `is_valid`, `coupon_id`, `code`, `discount_type`, numeric `discount_amount`, `final_amount`, nullable `message`.

Order and payment

- `OrderItemCreate`: required `menu_item_id:UUID`, `quantity:int` >0; optional unrestricted `selected_modifiers_snapshot:object[]`.
- `OrderCreateRequest`: product flow sends required `quote_id:UUID`. Legacy branch/items fields remain compatibility inputs but are not used by the product frontend.
- `OrderStatusUpdateRequest`: required `status:string`; optional `reason:string`. The service transition map is authoritative.
- `OrderItemResponse`: server `id`, `order_id`, `menu_item_id`, `item_name_snapshot`, `unit_price_snapshot`, `quantity`, nullable modifier snapshot, `created_at`.
- `OrderStatusLogResponse`: server `id`, `order_id`, nullable `previous_status`, `new_status`, `changed_by_user_id`, `created_at`.
- `OrderResponse`: server IDs, `checkout_quote_id`, `subtotal`, `discount_amount`, `tax_amount`, `delivery_fee`, `total_amount`, `currency`, status/cancellation data, trusted address snapshot, timestamps, items, and status logs.
- `PaymentIntentCreateRequest`: required `order_id:UUID`; optional compatibility amount/currency are ignored in favor of trusted order values.
- `PaymentIntentResponse`: `payment_id`, `order_id`, provider order ID, trusted amount/currency, provider, and optional public provider key ID.
- `PaymentVerifyRequest`: required `order_id:UUID`, `razorpay_order_id:string`, `razorpay_payment_id:string`, `razorpay_signature:string`.
- `PaymentVerifyResponse`: `success:boolean`, `payment_id`, `order_id`, `status:string`.
- `RefundCreateRequest`: required `amount:number` >0; `reason:string="ORDER_CANCELLED"` max 100; optional `idempotency_key`.
- `PaymentTransactionResponse`: server `id`, `payment_id`, `transaction_type`, nullable Razorpay IDs/signature, `amount`, `status`, nullable unrestricted `payload_snapshot`, `created_at`.
- `PaymentResponse`: server `id`, `order_id`, `customer_profile_id`, `amount`, `currency`, `created_at`, `transactions`.

Review, search, notification, subscription

- `ReviewCreate`: required `order_id:UUID`, `rating:int` 1–5; optional `comment` max 1000.
- `ReviewReply`: required `merchant_reply` 1–1000.
- `ReviewModerateRequest`: required `is_hidden:boolean`.
- `ReviewResponse`: server UUID IDs, rating/comment/reply/hidden flag, timestamps.
- `RestaurantRatingSummary`: `branch_id`, `average_rating`, `total_reviews`, `rating_distribution` object whose keys serialize as strings in JSON and values are integers.
- `SearchResultItem`: `entity_type`, `id`, `title`; optional description/branch/brand/price/veg/image/cuisines/distance; `is_available:boolean`.
- `SearchResponse`: `query`, `total`, and `restaurants`, `dishes`, `categories`, `results` arrays.
- `SuggestionItem`: `text`, `type`, optional `entity_id`; `SearchSuggestionsResponse`: `query`, `suggestions`.
- `NotificationSendRequest`: required `recipient:string`, `subject` 1–255, `body` min 1; `channel:string="EMAIL"` (ignored).
- `NotificationResponse`: server `id`, `user_id`, `channel`, `recipient`, nullable `subject`, `body`, `status`, nullable `vendor_message_id`, `created_at`.
- `SubscriptionCreateRequest`: `plan_tier:string="BASIC"`, `billing_cycle:string="MONTHLY"`; values are service-validated only as described above.
- `SubscriptionRenewalRequest`: optional `plan_tier`; `duration_months:int=1`, 1–24.
- `MerchantSubscriptionResponse`: IDs, `plan_tier`, `billing_cycle`, `status`, `commission_rate_percentage`, `max_branches`, `valid_until`, timestamps.
- `PlatformCommissionResponse`: IDs, `gross_order_amount`, rate, `commission_amount`, `merchant_net_payout`, `created_at`.
- `SubscriptionInvoiceResponse`: `invoice_number`, IDs, tier, amount, billing cycle, `valid_until`, `issued_at`.

Error contract

Application/domain errors use:

```
{
 "error_code": "MACHINE_READABLE_CODE",
 "message": "Human-readable message",
 "details": {},
 "correlation_id": "string-or-null"
}
```

Implemented status mappings are 400 domain invariant, 401 unauthorized, 403 forbidden, 404 entity not found, 409 conflict, 422 unprocessable entity, and 500 generic. Pydantic/path/query validation returns:

```
{
 "error_code": "VALIDATION_ERROR",
 "message": "Request validation failed",
 "details": {"errors": []},
 "correlation_id": "string-or-null"
}
```

The correlation ID comes from `CorrelationIdMiddleware`. A missing bearer token is raised by FastAPI's OAuth2 dependency before application auth handling and normally uses FastAPI's separate `{"detail":"Not authenticated"}` shape. Frontends should therefore handle both shapes. Some business validation, notably checkout validation, deliberately returns HTTP 200 with `is_valid=false` and an `issues` array.

Common codes include `INVALID_ACCESS_TOKEN`, `INVALID_TOKEN_TYPE`, `ROLE_NOT_AUTHORIZED`, `INVALID_UUID`, `*_NOT_FOUND`, `*_FORBIDDEN`, `EMAIL_ALREADY_EXISTS`, `DUPLICATE_GSTIN`, `CROSS_BRANCH_CART_CONFLICT`, `CROSS_BRANCH_ORDER_VIOLATION`, `ILLEGAL_STATUS_TRANSITION`, `INVALID_PAYMENT_SIGNATURE`, `INVALID_WEBHOOK_SIGNATURE`, and coupon-specific 422 codes.

Pagination and collection behavior

Pagination is offset-based and returns bare arrays without total/page metadata:

- Brands and branches: `skip` 0+, `limit` 1–100.
- Kitchens: `skip` 0+, `limit` 1–100.
- Search: `skip` 0+, `limit` 1–100; response includes `total`.
- Admin audit logs: only `limit` 1–500.
- Search suggestions: only `limit` 1–20.

All other list endpoints are unpaginated in the HTTP contract, including orders, inventory, reviews, addresses, favorites, notifications, coupons, commissions, invoices, banners, and feature flags.

Payment integration flow

1. Validate checkout and receive a short-lived quote, then create the order with `quote_id`.
2. Create an intent with the order ID. The backend ignores caller amount/currency and uses the persisted order values; repeated calls return the existing intent.
3. For `mock`, call `/payments/mock/complete`. For `razorpay`, open the provider UI with the returned provider order and public key ID.
4. Submit `order_id`, gateway order ID, gateway payment ID, and signature to `POST /payments/verify`.
5. On valid HMAC, the backend appends successful `VERIFIED` and `CAPTURED` transactions and returns `status="CAPTURED"`. Invalid HMAC appends failed `VERIFIED` and returns a 400 application error.
6. Razorpay may call `/payments/webhook`; signature is checked over the exact raw body. Duplicate captured payment IDs return `{"status":"ignored","reason":"DUPLICATE_WEBHOOK"}`. Unknown payment mappings return `{"status":"ignored","reason":"PAYMENT_NOT_FOUND"}`. `payment.captured` appends successful `CAPTURED`; `payment.failed` appends failed `CAPTURED`; other events return processed without mutation.
7. Authorized merchant roles can request a partial/full refund. The repository enforces cumulative refund ≤ captured amount.

`PAYMENT_PROVIDER=mock` is the development default. `razorpay` requires key ID, key secret, and webhook secret at startup. Ownership, provider order correspondence, trusted amount, duplicate verification/webhook IDs, refund caps, and refund idempotency keys are enforced. Gateway/webhook processing does not yet drive a separate order payment-state machine.

WebSocket contract (1)

Connect to:

```
ws(s)://<host>/api/v1/ws/branches/{branch_id}/orders
```

- `branch_id` must parse as UUID.
- Supply an access token using `?access_token=...` or the supported bearer subprotocol form. Only Kitchen Staff, Branch Manager, Merchant Owner, and Platform Admin claims are accepted, and non-admin users must own or be assigned the branch.
- The first accepted message is `RESYNC_REQUIRED` with branch, cursor, and poll URL. Client text frames have no application meaning and are consumed only to keep the connection open.
- On order creation, subscribers receive:

```
{"event":"ORDER_PLACED","order_id":"uuid","total_amount":123.45,"status":"PLACED"}
```

- On status transition, subscribers receive:

```
{"event":"ORDER_STATUS_CHANGED","order_id":"uuid","previous_status":"PLACED","status":"ACCEPTED"}
```

Connections are held in process memory by branch. There is no replay, sequence number, acknowledgment, heartbeat/ping protocol, resumption, delivery guarantee, cross-worker pub/sub, or message schema version. A reconnecting frontend must refresh state with `GET /orders?branch_id=...`.

End-to-end workflows

Customer

1. Register/login, store access and refresh tokens securely, and call `/auth/refresh` before/after access expiry.
2. Call `GET /customers/me` to ensure a profile exists; update it and create/select an address if needed.
3. Discover with brands, branches, search, menus, ratings, and coupons.
4. Add only one branch's items to the cart. Validate cart and checkout; treat `200 is_valid=false` as a blocked checkout.
5. Create the order from `quote_id`; the server atomically carries quote totals, coupon/address snapshots, and stock reservations into the order.
6. Create and verify payment separately.
7. Poll customer-scoped order details; branch WebSockets are restricted to authorized operations staff.
8. Cancel only while state permits; review only a completed order owned by the customer.

Merchant / kitchen

1. Accept an administrator invitation/provisioned Merchant Owner account, onboard legal/tax/bank data, submit the draft, and await admin approval.
2. Create brand, branch, kitchen, menu/category/items, and inventory.
3. Connect kitchen terminals to the branch WebSocket and refresh branch orders after reconnect.
4. Transition orders through `ACCEPTED`, `PREPARING`, `READY`, `COMPLETED`; quoted order creation reserves stock and cancellation restores it.
5. Read/refund payments and inspect subscription/commission/invoice views.

Merchant owners invite staff and maintain role/branch assignments. Every operational resource route rechecks those assignments server-side.

Platform admin

1. Use `PlatformAdmin` to approve/reject merchants, administer banners/coupons/flags, inspect audit logs/analytics, and create synthetic refund overrides.
2. Provision identities, issue invitations, revoke sessions, and moderate hidden reviews.
3. Treat synthetic refund overrides as administrative audit tools rather than gateway refunds.

Server-generated and snapshot fields

Clients must not manufacture response-only IDs or timestamps. Persistent entity IDs are generated as UUIDv7 by model defaults. `created_at` and `updated_at` are database-generated/maintained; soft-deleted records use an internal `deleted_at` not exposed by response schemas.

Other generated/derived fields:

- Auth: password hash, active/verified defaults, JWTs, expiry metadata.
- Customer: profile/customer IDs; default profile name on lazy creation; primary-address normalization is service-managed.
- Cart: cart/item IDs, current menu names/prices, subtotals, item count, expiry.
- Checkout: branch/kitchen identity, current-price lines, 5% tax, 30.00 delivery fee, coupon discount, final amount, issues.

- Coupon: uppercase code, platform flag (`branch_id is null`), calculated discount/final amount.
- Order: order/item/log IDs, item-name and unit-price snapshots, total, initial/transition statuses and logs.
- Payment: payment/transaction IDs, synthetic Razorpay IDs, HMAC result, transaction types/statuses and payload snapshots.
- Review: customer and branch IDs derived from the verified order.
- Subscription: commission rate, branch limit, validity, invoice number, commission/net payout.
- Admin: audit metadata, synthetic override ID, processing timestamp, aggregate analytics.

Quote-backed address and item price/name snapshots are server-derived. Selected modifier snapshots remain loosely structured compatibility data.

Residual limitations

The phase-seven audit leaves these non-blocking limitations explicit:

1. The default payment/email providers are local mocks. Razorpay configuration is startup-validated, but production deployments still need provider credentials, secret-manager injection, reconciliation, and operational retry monitoring.
2. The administrative refund override is an audited operational record, not a gateway refund. Use the tenant-scoped payment refund route for provider refunds.
3. WebSocket connections and event delivery are process-local. There is no cross-worker broker, replay log, acknowledgment, or heartbeat protocol; clients must honor `RESYNC_REQUIRED` and retain polling fallback.
4. Access JWTs are short-lived and stateless. Session revocation immediately blocks refresh but an already-issued access token remains valid until its 15-minute expiry.
5. Some collections remain bare, unpaginated arrays. Payment, inventory logs, admin merchants, and selected reports have bounded pagination/date windows.
6. Selected modifier snapshots remain loosely typed, and subscription invoices are synthesized rather than durable fiscal receipts.
7. Feature flags are persisted but are not a distributed configuration service and not every flag is wired into enforcement.
8. The Docker image applies Alembic migrations before one application process. A multi-replica production deployment should run migrations as one release job before scaling application replicas.
9. Framework-generated missing-bearer errors retain FastAPI's `detail` envelope; product clients normalize both that and the application error envelope.